



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Práctica 1
X-Route: Agente planificador de rutas

Búsqueda ciega, búsqueda informada y búsqueda local sobre el mapa real de la CDMX

Equipo:

Dávila Martínez Bryan · 2024630002
Hernández Alonso Diego Raymundo · 2022361124
Martínez Rios Edson Jared · 2024630284
Pallares Hernández Oscar · 2024630524

Grupo: 6CV2

Profesor: Andrés García Floriano

Fecha de entrega:

24/09/ 2026

Índice

1. Introducción	3
2. Entorno de trabajo y estructura del proyecto	3
3. Espacio de estados: el mapa de la CDMX	3
3.1 Descarga del grafo	4
3.2 Modelo formal	5
4. Fase 1: Búsqueda a ciegas	6
4.1 Implementación	6
4.2 Resultados	6
4.3 Verificación de accesibilidad	7
4.4 Avance de la frontera	8
4.5 Preguntas de la Fase 1	9
5. Fase 2: Búsqueda informada	10
5.1 Heurísticas	10
5.2 A* y Greedy Best-First	10
5.3 Resultados	10
5.4 Admisibilidad de las heurísticas	11
5.5 Visualización con folium	12
5.6 Preguntas de la Fase 2	14
6. Fase 3: Búsqueda local	14
6.1 Definición del problema	14
6.2 Simulated Annealing	15
6.3 Algoritmo Genético	15
6.4 Resultados	15
6.5 Convergencia	16
6.6 Experimento de la temperatura inicial	17
6.7 Preguntas de la Fase 3	17
7. Pruebas unitarias	18
8. Conclusiones	19
9. Referencias	19

1. Introducción

El objetivo de esta práctica fue construir **X-Route**, un agente que planea rutas de entrega de "última milla" sobre un mapa real de la zona norte y centro de la Ciudad de México. En lugar de probar los algoritmos en laberintos o grafos de juguete, los corrimos sobre las calles reales descargadas de OpenStreetMap, con sus sentidos de circulación, avenidas diagonales y calles cerradas incluidas.

El agente resuelve tres problemas, uno por fase:

- **Fase 1 (búsqueda a ciegas):** saber qué destinos son alcanzables desde el depósito y encontrar caminos con BFS, DFS y UCS, sin usar coordenadas.
- **Fase 2 (búsqueda informada):** usar la ubicación geográfica para guiar la búsqueda con A* y Greedy Best-First, probando tres heurísticas distintas.
- **Fase 3 (búsqueda local):** dado un camión con 15 paquetes, decidir en qué orden entregarlos para recorrer la menor distancia, usando Simulated Annealing y un Algoritmo Genético.

Una regla que nos pusimos desde el inicio fue mantener el código lo más simple posible. Todos los algoritmos están escritos desde cero con la biblioteca estándar de Python (heapq, collections.deque, random, math). Las librerías osmnx y networkx solo las usamos para descargar y leer el mapa, nunca para buscar caminos. Esto nos obligó a entender bien cada algoritmo y además nos sirvió para comparar nuestros resultados contra los de networkx y confirmar que estaban bien.

En este reporte explicamos cómo modelamos el problema, cómo implementamos cada algoritmo, los resultados que obtuvimos y las respuestas a las preguntas planteadas en cada fase.

2. Entorno de trabajo y estructura del proyecto

Trabajamos en Windows con Python 3.12.6 dentro de un entorno virtual (venv). Las dependencias son las que pedía la práctica: osmnx, networkx, folium, matplotlib y pytest, más jupyter. Todo está en requirements.txt, así que para instalar basta con:

```
py -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

La estructura del repositorio quedó así:

Tabla 1. Estructura de carpetas y archivos principales.

Ruta	Contenido
src/descargar_grafo.py	Descarga el mapa de OpenStreetMap y lo guarda en data/
src/mapa.py	Carga el grafo y lo convierte a diccionarios simples; lugares de prueba
src/fase1.py	BFS, DFS, UCS y verificación de accesibilidad
src/snapshots_fase1.py	Imágenes del avance de la frontera

Ruta	Contenido
src/fase2.py	Heurísticas h1, h2, h3; A* y Greedy
src/admisibilidad.py	Validación empírica de admisibilidad
src/mapas_fase2.py	Mapas interactivos con folium
src/fase3.py	Definición del TSP, Simulated Annealing y Algoritmo Genético
src/graficas_fase3.py	Convergencia y experimento de T0
src/ruta_entregas_fase3.py	Mapa del recorrido del camión
tests/	28 pruebas unitarias con pytest
data/, imagenes/, mapas/	Grafo descargado, figuras y mapas HTML generados
README.md, ai_log.md	Instrucciones de uso y registro de uso de IA

Todos los scripts se corren desde la carpeta raíz, por ejemplo `python src/fase1.py`. Esto es porque la ruta al grafo (`data/grafos_cdmx.graphml`) es relativa. Para las pruebas agregamos un `pytest.ini` con `pythonpath = src`, así las pruebas pueden importar los módulos directamente.

3. Espacio de estados: el mapa de la CDMX

3.1 Descarga del grafo

Para descargar el mapa usamos `graph_from_point` de `osmnx` con un centro en (19.450, -99.137), que queda más o menos entre el Zócalo y La Raza, y un radio de 3 km. Elegimos el tipo de red "drive" porque nuestro agente es un camión de reparto y solo debe usar calles para autos, respetando el sentido de cada una.

Decidimos usar un radio de 3 km después de pensar en el tamaño del grafo. Con 5 km o tomando alcaldías completas el grafo crece mucho (decenas de miles de nodos) y algoritmos como DFS y UCS se vuelven lentos en Python. Con 3 km cubrimos el Centro Histórico, Tlatelolco, Guerrero, parte de la Gustavo A. Madero y la Morelos, que es bastante para que el problema sea interesante.

```
(.venv) PS C:\Inteligencia Artificial ESCOM\Prac1> python src/descargar_grafo.py
Versión de osmnx: 2.1.1
Descargando grafo... (puede tardar 1-2 minutos)
Nodos (intersecciones): 5235
Aristas (calles): 11642
Grafo guardado en: data/grafos_cdmx.graphml
Imagen guardada en: data/grafos_cdmx.png
```

Figura 1. Salida de `descargar_grafo.py`.



Figura 2. Grafo vial descargado (5,235 intersecciones y 11,642 calles). Se distinguen Reforma en diagonal, Insurgentes Norte y el Circuito Interior.

3.2 Modelo formal

El mapa lo tratamos como un grafo dirigido $G = (V, E, w)$ donde:

- **V (nodos):** las intersecciones de calles. Tenemos 5,235.
- **E (aristas):** los tramos de calle entre dos intersecciones. Como el grafo es dirigido, una calle de un solo sentido es una sola arista y una de doble sentido son dos.
- **w (pesos):** la longitud de cada tramo en metros (atributo length de osmnx).

En lugar de trabajar directamente sobre el MultiDiGraph de networkx, en mapa.py lo convertimos a dos diccionarios de Python: `vecinos = {nodo: [(vecino, metros), ...]}` y `coords = {nodo: (lat, lon)}`. Así los algoritmos solo usan listas y diccionarios, son más fáciles de explicar y los podemos probar con grafos pequeños hechos a mano. Cuando había dos calles paralelas entre los mismos nodos nos quedamos con la más corta, porque es la que tomaría un camión. Por eso el grafo final tiene 11,625 aristas en vez de 11,642.

El depósito es la intersección más cercana al centro del mapa (quedó en Tlatelolco, a 119 m del punto pedido). Para las pruebas de las fases 1 y 2 elegimos seis lugares conocidos como destinos, lo que ayuda mucho a interpretar los resultados. Para convertir cada lugar a un nodo buscamos la intersección más cercana usando la distancia Haversine; es una búsqueda lineal sobre los 5 mil nodos, pero es rápida y nos evitó instalar scikit-learn.

```
(.venv) PS C:\Inteligencia Artificial ESCOM\Prac1> python src/mapa.py
Nodos: 5235
Aristas (sin paralelas): 11625
Nodos sin calles de salida: 23

Depósito -> nodo 268615567 (a 119 m del punto pedido)
Bellas Artes -> nodo 2180585469 (a 71 m)
Garibaldi -> nodo 269994753 (a 41 m)
Monumento a la Revolución -> nodo 4521825356 (a 75 m)
La Raza -> nodo 1409192275 (a 51 m)
Tepito -> nodo 268620414 (a 70 m)
Zócalo -> nodo 2113341940 (a 69 m)
```

Figura 3. Salida de mapa.py: resumen del grafo y nodo asignado a cada lugar.

4. Fase 1: Búsqueda a ciegas

En esta fase los algoritmos no tienen idea de dónde está el destino, van revisando vecinos nomás. Los tres regresan la info en el mismo formato (camino, metros, saltos, nodos expandidos, tamaño máximo de la frontera, tiempo en ms y orden de expansión), esto porque necesitábamos que las tablas comparativas cuadraran bien, incluyendo las de la Fase 2. .

4.1 Implementación

BFS usa una cola FIFO (`collections.deque`, porque sacar del frente de una lista normal con `pop(0)` es lento). Primero revisa los nodos a un salto, luego los de dos, y así se va extendiendo, entonces el primer camino que llega al destino ya es el de menos saltos. Un diccionario de padres guarda de dónde llegamos a cada nodo, y de paso nos sirve como conjunto de visitados.

DFS lo hicimos iterativo, con una lista funcionando como pila LIFO. Optamos por esto porque con miles de nodos una versión recursiva se podía pasar del límite de recursión de Python. Aquí sí o sí necesitamos un conjunto de visitados, porque las calles tienen ciclos (como dar la vuelta a la manzana) y sin ese control el algoritmo se quedaría dando vueltas sin parar. En la pila guardamos pares (nodo, padre), ya que un nodo puede meterse varias veces desde distintos vecinos, y el padre que realmente importa es el de cuando se expande de verdad.

UCS usa `heapq` como cola de prioridad, con tuplas (costo, nodo), para que siempre salga primero el nodo con menos metros acumulados. Cuando encontramos un camino más barato hacia un nodo que ya estaba metido en la cola, simplemente agregamos una copia nueva y dejamos que la vieja se ignore cuando le toque salir (esto se conoce como eliminación perezosa), que resulta más simple que andar buscándola y modificándola dentro del heap.

En los tres algoritmos revisamos si ya llegamos a la meta justo cuando sacamos el nodo de la frontera, no cuando lo metemos, para que el conteo de nodos expandidos fuera comparable entre todos. También comparamos BFS y UCS contra `networkx` (`shortest_path` y `dijkstra`) nada más como verificación, y coincidieron exactamente en saltos y metros en los seis pares que probamos.

4.2 Resultados

Tabla 2. Resultados de la Fase 1 desde el depósito (metros / saltos / nodos expandidos / frontera máxima).

Destino	BFS	DFS	UCS
Bellas Artes	2,897 / 20 / 1,223 / 147	135,644 / 1,334 / 3,756 / 1,009	2,655 / 25 / 1,626 / 150
Garibaldi	1,995 / 15 / 587 / 128	133,292 / 1,316 / 3,951 / 1,043	1,876 / 19 / 710 / 101
Monumento a la Revolución	3,140 / 19 / 1,106 / 146	70,637 / 670 / 936 / 480	2,669 / 26 / 1,640 / 150
La Raza	3,765 / 30 / 2,850 / 191	97,226 / 954 / 1,901 / 714	2,795 / 31 / 1,818 / 153
Tepito	1,956 / 19 / 1,008 / 146	116,583 / 1,160 / 3,473 / 894	1,813 / 24 / 653 / 94
Zócalo	2,885 / 24 / 1,829 / 191	126,929 / 1,266 / 3,663 / 966	2,634 / 27 / 1,602 / 150

Tabla 3. Tiempo de ejecución de la Fase 1 en milisegundos.

Destino	BFS	DFS	UCS
Bellas Artes	0.93	2.56	1.75
Garibaldi	0.31	2.60	0.65
Monumento a la Revolución	0.49	0.56	1.95
La Raza	1.48	1.16	1.71
Tepito	0.52	2.19	0.74
Zócalo	0.90	2.40	1.54

Lo primero que llama la atención es DFS: para llegar a Bellas Artes, que queda a unos 2 km, terminó encontrando un camino de ¡135 km! No es ningún error, así se comporta el algoritmo: agarra la primera calle que se le atraviesa y sigue derecho lo más que puede, y solo se devuelve cuando ya no tiene por dónde seguir. El camino que arma depende totalmente del orden en que están guardados los vecinos.

Entre BFS y UCS el patrón se ve clarísimo: BFS siempre gana en saltos y UCS siempre gana en metros. En La Raza la diferencia llega a casi un kilómetro (3,765 m contra 2,795 m), nomás por un salto menos. En cuanto a tiempo, los dos se mueven entre 1 y 2 ms; UCS suele tardar un poquito más porque cada operación del heap cuesta $O(\log n)$, mientras que la cola de BFS es $O(1)$.

```
(.venv) PS C:\Inteligencia Artificial ESCOM\Prac1> python src/fase1.py
```

Alg	Destino	Metros	Salto	Expand.	Front.max	ms
BFS	Bellas Artes	2897	20	1223	147	0.93
DFS	Bellas Artes	135644	1334	3756	1009	2.56
UCS	Bellas Artes	2655	25	1626	150	1.75
BFS	Garibaldi	1995	15	587	128	0.31
DFS	Garibaldi	133292	1316	3951	1043	2.60
UCS	Garibaldi	1876	19	710	101	0.65
BFS	Monumento a la Revolución	3140	19	1106	146	0.49
DFS	Monumento a la Revolución	70637	670	936	480	0.56
UCS	Monumento a la Revolución	2669	26	1640	150	1.95
BFS	La Raza	3765	30	2850	191	1.48
DFS	La Raza	97226	954	1901	714	1.16
UCS	La Raza	2795	31	1818	153	1.71
BFS	Tepito	1956	19	1008	146	0.52
DFS	Tepito	116583	1160	3473	894	2.19
UCS	Tepito	1813	24	653	94	0.74
BFS	Zócalo	2885	24	1829	191	0.90
DFS	Zócalo	126929	1266	3663	966	2.40
UCS	Zócalo	2634	27	1602	150	1.54

Figura 4. Salida de fase1.py con los tres algoritmos.

4.3 Verificación de accesibilidad

Para saber qué destinos se pueden alcanzar hicimos una función `nodos_alcanzables()`, que básicamente es un BFS pero sin destino fijo: nomás sigue corriendo hasta que se vacía la cola y al final regresa todo lo que sí se puede alcanzar. Ya con eso calculado una sola vez, checar cualquier entrega es tan fácil como ver si está dentro de ese conjunto.

Desde el depósito se alcanzan 5,169 de los 5,235 nodos, o sea que quedan 66 que no se pueden alcanzar. Todos están en la orilla del mapa, en calles de un solo sentido que en la vida real siguen más allá del radio de 3 km que usamos; como ahí cortamos el mapa, pues el camión ya no tiene por dónde meterse. Los seis lugares que usamos de prueba sí se pueden alcanzar sin bronca. Y algo curioso: cuando le pedimos a UCS que buscara un destino inalcanzable, expandió exactamente 5,169 nodos antes de darse por vencido, porque para poder decir "no hay camino" una búsqueda ciega tiene que revisar absolutamente todo lo que sí alcanza. Por eso más vale checar primero si algo es alcanzable antes de ponerte a planear rutas

4.4 Avance de la frontera

Para las imágenes no modificamos los algoritmos. Usamos la lista orden_expansion: los nodos expandidos al k% son los primeros k de esa lista, y la frontera son sus vecinos que todavía no se expanden. Dibujamos las tres búsquedas hacia La Raza al 25%, 50% y 100%.

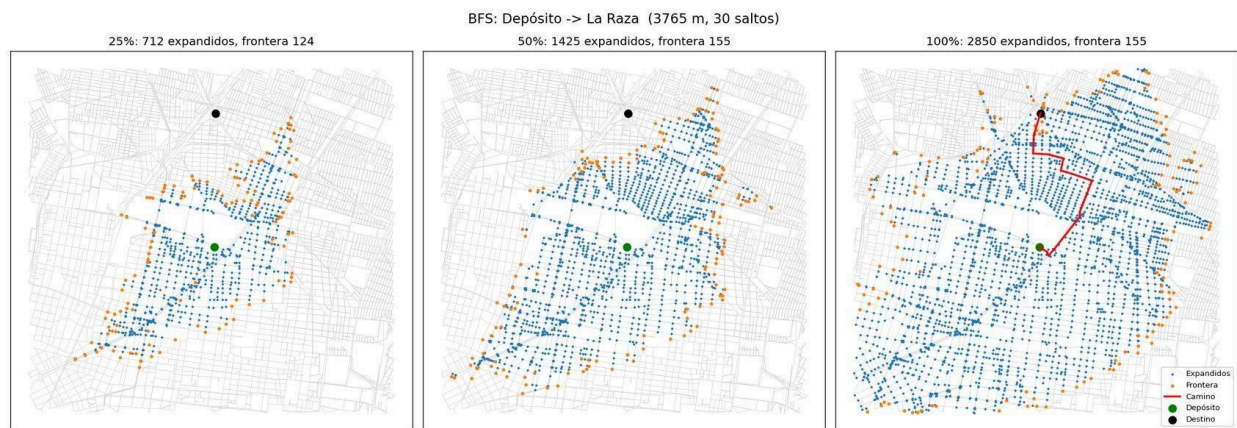


Figura 5. BFS hacia La Raza. Crece como una mancha por número de calles; se estira hacia las zonas de cuadradas chicas.

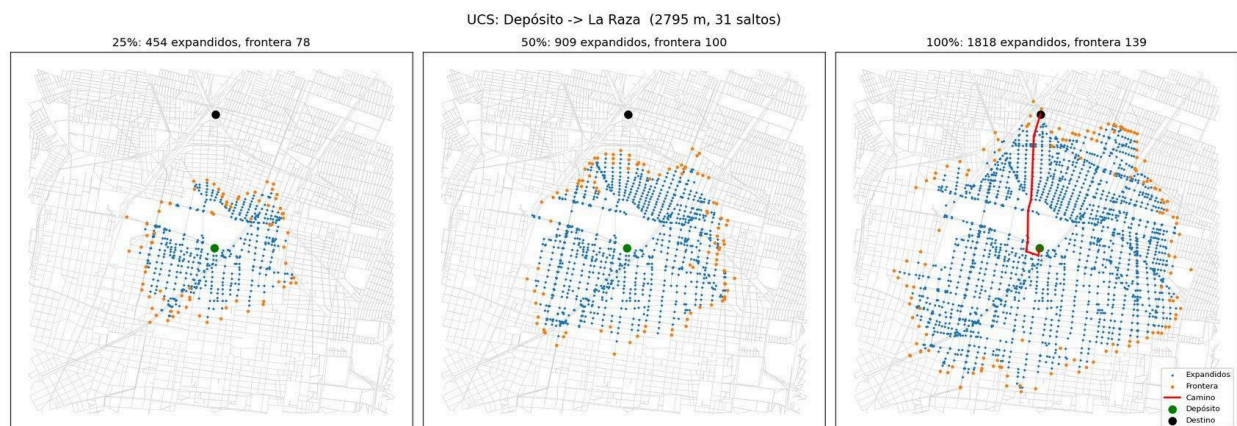


Figura 6. UCS hacia La Raza. Crece de forma pareja por distancia en metros y su camino va casi recto al norte.

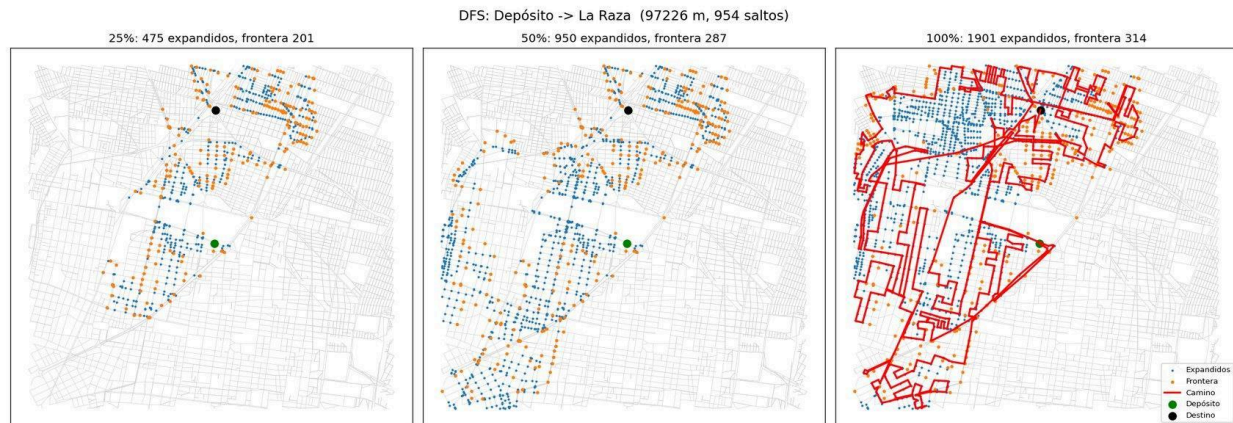


Figura 7. DFS hacia La Raza. No forma una mancha, se dispersa en "culebra" y su camino recorre medio mapa.

Una aclaración: el número de "frontera" que aparece en cada panel es una reconstrucción visual con nodos únicos y no coincide exactamente con la frontera máxima de la tabla. En DFS la pila real es más grande porque puede tener nodos repetidos. Para las métricas siempre usamos los números de la tabla.

4.5 Preguntas de la Fase 1

¿Por qué BFS garantiza el camino con menos saltos pero no el de menor distancia?

Porque BFS explora por niveles: primero se avienta a revisar todos los nodos a 1 salto, luego los de 2, y así. Entonces cuando por fin llega al destino, cualquier otro camino que exista va a tener igual o más saltos, ya no le gana nadie. El detalle es que BFS nunca se fija en los metros; para él una cuadra de 50 m y una avenida de 800 m valen exactamente lo mismo, un salto nada más. Por eso un camino con pocas calles bien largas le puede ganar a uno con más calles cortas, aunque ese mida menos en total. Y así nos pasó en los seis pares que probamos: BFS siempre salió con menos saltos, y UCS siempre con menos metros

¿En qué estructuras de grafo es preferible DFS? ¿Aplica en este caso urbano?

DFS conviene cuando el problema es la memoria y el grafo está bien profundo, cuando con cualquier solución te conformas (no te importa si es la mejor o no), o en cosas tipo árbol o laberinto donde no hay muchos caminos alternativos, por ejemplo para ver si hay una salida, detectar ciclos o hacer un ordenamiento topológico. Pero en un mapa urbano no sirve para planear rutas: la ciudad está llena de ciclos y caminos alternativos, y DFS se va por el primero que se encuentra, así que terminas con rutas absurdas de 70 hasta 135 km. Donde sí le podría entrar es en checar accesibilidad, porque ahí lo único que importa es si hay camino o no, aunque nosotros para eso mejor usamos BFS.

¿Cuándo UCS y BFS producen exactamente el mismo resultado?

Cuando todas las aristas cuestan lo mismo. Si cada calle midiera, por decir, 100 m parejo, el costo de cualquier camino sería $100 \times \text{saltos}$, entonces minimizar metros y minimizar saltos vendría siendo lo mismo, y ahí sí UCS se comportaría igualito que BFS. También se puede dar por pura casualidad en algún par específico, si resulta que el camino con menos saltos también es el más corto, pero en ninguno de nuestros seis pares se dio, porque las calles de la ciudad miden cosas bien distintas entre sí.

5. Fase 2: Búsqueda informada

En esta fase los algoritmos de búsqueda informada utilizan conocimientos adicionales, denominados heurísticas, en nuestro caso se conocen las coordenadas del destino y las usamos para estimar cuánto falta, con una función heurística $h(n)$. Todas nuestras heurísticas regresan metros, para que se puedan sumar con el costo real $g(n)$, que también está en metros.

5.1 Heurísticas

h1, Euclidiana. Distancia en línea recta sobre un plano. Primero pasamos la diferencia de latitud y longitud a metros y luego aplicamos Pitágoras. La longitud se multiplica por el coseno de la latitud porque los meridianos se van juntando conforme te alejas del ecuador:

$$dx = R \cdot \Delta\lambda \cdot \cos(\varphi), \quad dy = R \cdot \Delta\varphi, \quad h1 = \sqrt{dx^2 + dy^2}$$

h2, Haversine. Distancia en línea recta sobre la esfera terrestre, con $R = 6,371$ km:

$$a = \sin^2(\Delta\varphi/2) + \cos \varphi_1 \cdot \cos \varphi_2 \cdot \sin^2(\Delta\lambda/2), \quad h2 = 2R \cdot \arcsin(\sqrt{a})$$

h3, Personalizada (distancia + giros). Es Haversine más una penalización de 50 m si probablemente hay que doblar. La idea es que en una cuadrícula de calles, si el destino está en línea recta al norte, sur, este u oeste podrías llegar sin dar vuelta, pero si está en diagonal tienes que doblar al menos una vez. Con Atan2 sacamos el ángulo hacia el destino y medimos cuánto se aleja del eje más cercano; si se aleja más de 20° , contamos un giro. Desde el principio sabíamos que esta heurística podía fallar ya que el costo real solo cuenta metros y doblar una esquina no cuesta metros extra, la dejamos así para poder medir qué pasa con una heurística no admisible.

5.2 A* y Greedy Best-First

A* es no es igual a UCS pero están estrechamente relacionados, la prioridad en el heap es $f(n) = g(n) + h(n)$ en lugar de solo $g(n)$. Así prefiere los nodos que se acercan al destino. **Greedy** usa solo $f(n) = h(n)$: siempre expande el nodo que parece más cercano al destino, sin importar cuánto costó llegar ahí. Como no compara costos, basta con marcar cada nodo la primera vez que se descubre, igual que en BFS.

5.3 Resultados

Tabla 4. Metros del camino encontrado. Resaltado: el único caso donde A* no dio el óptimo.

Destino	UCS	A* h1	A* h2	A* h3	Greedy h2
Bellas Artes	2,655	2,655	2,655	2,655	3,257
Garibaldi	1,876	1,876	1,876	1,876	2,148
Monumento a la Revolución	2,669	2,669	2,669	2,679	3,340
La Raza	2,795	2,795	2,795	2,795	3,149
Tepito	1,813	1,813	1,813	1,813	1,914
Zócalo	2,634	2,634	2,634	2,634	3,242

Tabla 5. Nodos expandidos.

Destino	UCS	A* h1	A* h2	A* h3	Greedy h2
Bellas Artes	1,626	275	275	269	44
Garibaldi	710	173	173	168	58
Monumento a la Revolución	1,640	148	148	135	40
La Raza	1,818	55	55	55	58
Tepito	653	54	54	55	23
Zócalo	1,602	162	162	162	42

A* con h1 y h2 descubrió exactamente el mismo camino óptimo que UCS en los seis pares, pero expandiendo entre 3 y 33 veces menos nodos (en La Raza, 55 contra 1,818). Greedy expande todavía menos, pero en los seis pares dio caminos más largos, entre 101 m y 671 m de más. encontramos que en La Raza, donde Greedy expandió más nodos que A* (58 contra 55): Greedy no siempre es más rápido, si se equivoca de dirección tiene que explorar de más para salir del error.

Con h3 pasó lo que esperábamos: en Monumento a la Revolución A* dio 2,679 m, 10 m más que el óptimo, a cambio de expandir un poco menos (135 contra 148). Es el intercambio típico de una heurística mas agresiva si es más rápida pero sin garantía de ser mejor..

```
(.venv) PS C:\Inteligencia Artificial ESCOM\Prac1> python src/fase2.py
```

Alg	Destino	Metros	Salto	Expand.	Front.max	ms
UCS	Bellas Artes	2655	25	1626	150	2.90
A*-h1	Bellas Artes	2655	25	275	50	0.77
A*-h2	Bellas Artes	2655	25	275	50	1.14
A*-h3	Bellas Artes	2655	25	269	52	1.06
Gr-h2	Bellas Artes	3257	34	44	27	0.15
UCS	Garibaldi	1876	19	710	101	1.01
A*-h1	Garibaldi	1876	19	173	36	0.44
A*-h2	Garibaldi	1876	19	173	36	0.43
A*-h3	Garibaldi	1876	19	168	37	0.59
Gr-h2	Garibaldi	2148	22	58	21	0.18
UCS	Monumento a la Revolución	2669	26	1640	150	2.40
A*-h1	Monumento a la Revolución	2669	26	148	65	0.41
A*-h2	Monumento a la Revolución	2669	26	148	65	0.51
A*-h3	Monumento a la Revolución	2679	32	135	61	0.54
Gr-h2	Monumento a la Revolución	3340	36	40	31	0.16
UCS	La Raza	2795	31	1818	153	2.46
A*-h1	La Raza	2795	31	55	42	0.24
A*-h2	La Raza	2795	31	55	42	0.18
A*-h3	La Raza	2795	31	55	42	0.26
Gr-h2	La Raza	3149	37	58	39	0.19
UCS	Tepito	1813	24	653	94	0.79
A*-h1	Tepito	1813	24	54	23	0.19
A*-h2	Tepito	1813	24	54	23	0.14
A*-h3	Tepito	1813	24	55	24	0.21
Gr-h2	Tepito	1914	21	23	20	0.09
UCS	Zócalo	2634	27	1602	150	1.95
A*-h1	Zócalo	2634	27	162	48	0.67
A*-h2	Zócalo	2634	27	162	48	0.52
A*-h3	Zócalo	2634	27	162	49	0.63
Gr-h2	Zócalo	3242	36	42	29	0.15

Figura 8. Salida de fase2.py con UCS, A* (h1, h2, h3) y Greedy.

5.4 ¿Son confiables las heurísticas?

Demostración teórica para h1 y h2. Una heurística es admisible si nunca exagera: su estimación de lo que falta por recorrer jamás es mayor al costo real: $h(n) \leq h^*(n)$. Cualquier camino por calles entre dos puntos es una curva que los une y ninguna curva puede ser más corta que la línea recta (que es lo que mide Haversine). Como además osmnx calcula la longitud de cada calle siguiendo su geometría, la suma de las calles de cualquier camino es

mayor o igual a la distancia en línea recta. Por lo tanto $h2 \leq h^*$. Para $h1$ pasa lo mismo porque a 3 km la proyección plana coincide con Haversine a menos de un milímetro.

Validación. En `admisibilidad.py` calculamos el costo real $h^*(n)$ de todos los nodos hacia cada uno de los seis destinos. Lo que hicimos fue invertir el grafo: la distancia de cualquier nodo hacia el destino es igual a la distancia del destino hacia ese nodo en el grafo con las calles al revés. Así, con una sola búsqueda (Dijkstra sin destino) desde cada destino obtuvimos los 5 mil valores, en vez de correr una búsqueda por nodo. En total revisamos 31,050 casos. También probamos $h2$ multiplicada por $k > 1$ para la pregunta correspondiente.

Tabla 6. Validación empírica de admisibilidad (31,050 casos por heurística).

Heurística	Sobreestima	%	Peor exceso (m)	h/h^* promedio	¿Admisible?
h1 Euclidiana	0	0.00%	0.0	0.745	Sí
h2 Haversine	0	0.00%	0.0	0.745	Sí
h3 Personalizada	33	0.11%	50.0	0.754	No
$h2 \times 1.2$	6,405	20.63%	987.2	0.894	No
$h2 \times 1.5$	25,252	81.33%	2,632.5	1.118	No
$h2 \times 2.0$	30,108	96.97%	5,791.8	1.490	No

$h1$ y $h2$ nunca sobreestimaron, y en promedio estiman el 74.5% del costo real (el otro 25% son las vueltas que obligan las calles). $h3$ sobreestimó en 33 casos, muy pocos, pero suficientes para que deje de ser admisible y para explicar los 10 m extra en Monumento a la Revolución.

```
(.venv) PS C:\Inteligencia Artificial ESCOM\Prac1> python src/admisibilidad.py
Heurística    Revisados  Sobreestima  %    Peor exceso (m)  h/h* prom.  Admisible
h1 Euclidiana    31050      0           0.00%    0.0             0.745      Sí
h2 Haversine     31050      0           0.00%    0.0             0.745      Sí
h3 Personalizada 31050      33          0.11%    50.0            0.754      NO
h2 x 1.2         31050      6405        20.63%   987.2           0.894      NO
h2 x 1.5         31050      25252       81.33%   2632.5          1.118      NO
h2 x 2.0         31050      30108       96.97%   5791.8          1.490      NO
```

Figura 9. Salida de `admisibilidad.py`.

5.5 Visualización con folium

Generamos un mapa HTML interactivo por destino con la ruta de A^* en azul y la de Greedy en rojo, cada una en su propia capa para poder visualizar la que quisiéramos. Al principio usamos el fondo de OpenStreetMap, pero al abrir los mapas como archivo local los servidores de OSM respondían 403 Access blocked, porque piden saber desde qué sitio web se hace la petición. Probamos CartoDB, pero ahora pide clave de API, así que al final usamos el mapa de calles de Esri.

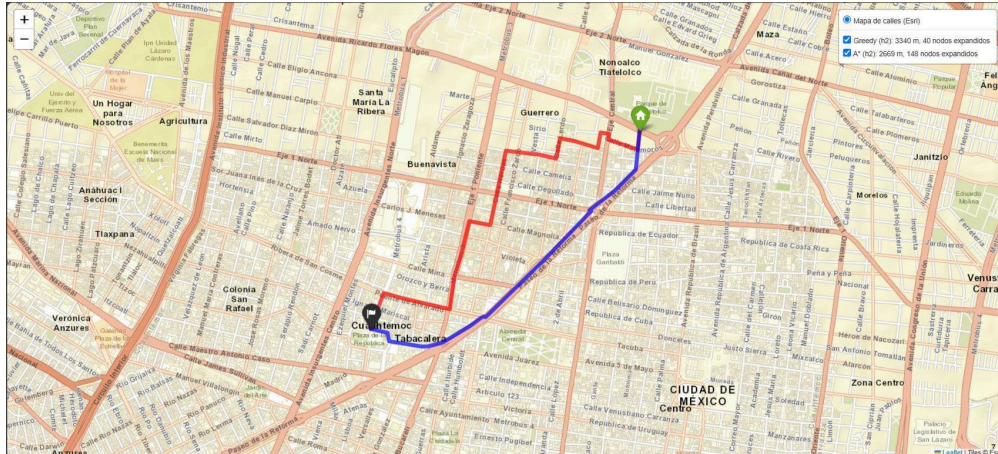


Figura 10. Monumento a la Revolución: A* (azul, 2,669 m) toma Paseo de la Reforma; Greedy (rojo, 3,340 m) se mete a la colonia Guerrero haciendo escalones.

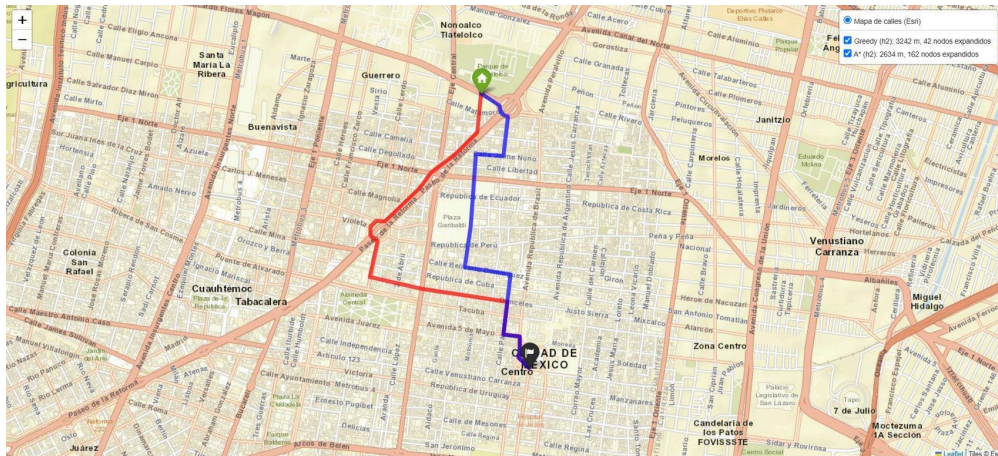


Figura 11. Zócalo: A* (2,634 m) contra Greedy (3,242 m).

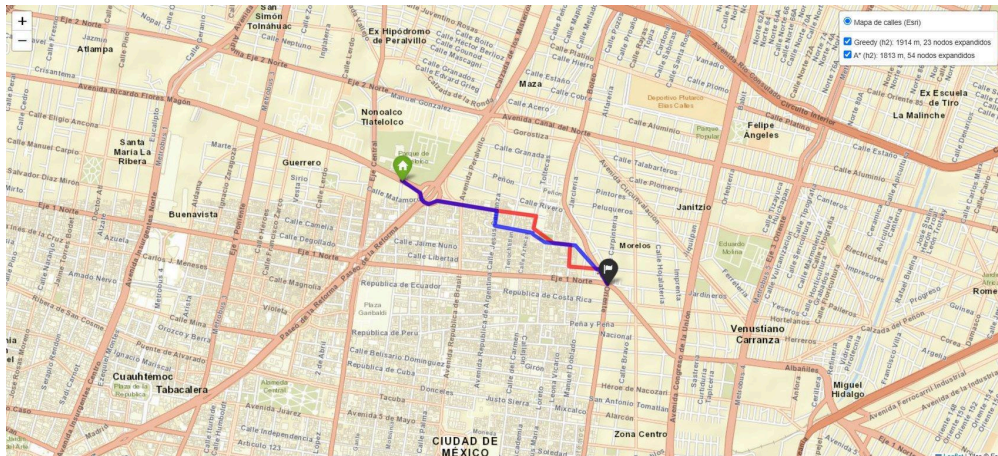


Figura 12. Tepito: aquí Greedy casi acierta, solo 101 m de más (1,914 m contra 1,813 m).

5.6 Preguntas de la Fase 2

¿Por qué Haversine es más precisa que la Euclidiana aquí?

En teoría, porque la Tierra no es plana: la distancia Euclidiana sobre latitud y longitud trata al mapa como un plano, mientras que Haversine mide el arco sobre la superficie de la esfera. Además, en grados reales, un grado de longitud no mide lo mismo que uno de latitud, buscando en fuentes encontramos que a la latitud de la CDMX, un grado de longitud mide aproximadamente 94% de uno de latitud.

Muestra un ejemplo donde Greedy falle en encontrar el camino más corto.

Greedy falló en los seis pares, pero el caso más claro es Monumento a la Revolución (Figura 10): encontró un camino de 3,340 m contra 2,669 m del óptimo, 671 m de más. A* sale del depósito y toma Paseo de la Reforma, una avenida en diagonal que va casi directo al destino. Greedy, en cada paso, escoge la intersección que se ve más cerca del Monumento en línea recta, y eso lo mete a las calles de la colonia Guerrero, donde tiene que ir en escalones. Nunca ve que Reforma, aunque al inicio no apunta exactamente al destino, era mucho más directa, porque ignora lo que ya recorrió ($g(n)$).

Si multiplicas h por una constante $k > 1$, ¿sigue siendo admisible?

En general no. Si h es admisible, $h \leq h^*$, pero $k \cdot h$ puede pasarse de h^* en los nodos donde h ya estaba cerca del valor real, que es justo lo que pasa cuando el camino real es casi una línea recta. Lo comprobamos en la Tabla 6: con $k = 1.2$ ya sobreestima en el 20.6% de los nodos, y con $k = 2$ en casi el 97%. Solo seguiría siendo admisible si todos los valores de h fueran menores a h^*/k , lo cual no se puede garantizar. Aun así, multiplicar h por k (lo que se conoce como A* ponderado) se usa en la práctica cuando se prefiere velocidad sobre optimalidad: expande menos nodos y el camino encontrado cuesta como máximo k veces el óptimo.

6. Fase 3: Búsqueda local

6.1 Definición del problema

El camión sale del depósito, realiza 15 entregas y regresa, formando un ciclo cerrado (TSP).

- Entregas: Se eligen 15 nodos aleatorios con `random.seed(42)`, asegurando que sean accesibles desde y hacia el depósito.
- Matriz de distancias: Se calcula la distancia mínima entre los puntos usando A* con h_2 , realizando 240 búsquedas.
- Ruta: Representa el orden de las entregas; por ejemplo, $[3, 1, 2]$ significa depósito $\rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow$ depósito.
- Asimetría: Ir de $A \rightarrow B$ puede tener una distancia diferente que $B \rightarrow A$ debido al sentido de las calles, lo cual afecta a 2-opt.

Como referencia, visitar las entregas en orden del 1 al 15 produce una distancia de 61,835 m.

```

(.venv) PS C:\Inteligencia Artificial ESCOM\Prac1> python src/fase3.py
Matriz 16x16 calculada en 1.4 s

Entregas elegidas:
0. Depósito      nodo 268615567 (19.4490147, -99.1365689)
1. Entrega 1     nodo 278211209 (19.4508552, -99.1462167)
2. Entrega 2     nodo 268615854 (19.4243142, -99.1397636)
3. Entrega 3     nodo 271346761 (19.4456414, -99.1087818)
4. Entrega 4     nodo 271047330 (19.4577861, -99.1208072)
5. Entrega 5     nodo 271035351 (19.4434483, -99.1212046)
6. Entrega 6     nodo 270290659 (19.4574434, -99.1623012)
7. Entrega 7     nodo 270114896 (19.4387815, -99.1411146)
8. Entrega 8     nodo 6106994175 (19.4536181, -99.130803)
9. Entrega 9     nodo 270107674 (19.4341873, -99.1566475)
10. Entrega 10   nodo 8253643131 (19.4462997, -99.1376764)
11. Entrega 11   nodo 386285456 (19.4713635, -99.1107224)
12. Entrega 12   nodo 268900107 (19.4631659, -99.1486979)
13. Entrega 13   nodo 268639106 (19.4260892, -99.1352539)
14. Entrega 14   nodo 270110512 (19.4418919, -99.149688)
15. Entrega 15   nodo 271032317 (19.4465619, -99.1198487)

Ruta en orden 1..15: 61835 m

```

Figura 13. Entregas elegidas con la semilla 42.

6.2 Simulated Annealing

Partimos de una ruta aleatoria y usamos 2-opt para generar rutas vecinas. Si la nueva ruta es mejor, se acepta; si es peor, puede aceptarse según la temperatura, permitiendo explorar más soluciones.

Usamos $T_0 = 1000$, $\alpha = 0.999$ y $T_{\min} = 0.1$, dando aproximadamente 9,200 iteraciones. La temperatura disminuye progresivamente para reducir la aceptación de rutas peores.

Como la matriz es asimétrica, después de aplicar 2-opt recalculamos la distancia completa de la ruta para obtener resultados correctos.

6.3 Algoritmo Genético

Usamos 100 rutas aleatorias durante 200 generaciones. En cada generación:

- Elitismo: las 2 mejores rutas pasan directamente a la siguiente generación.
- Selección por torneo ($k=3$): se comparan 3 rutas al azar y se selecciona la mejor.
- Cruza OX (0.9): combina dos rutas sin repetir entregas.
- Mutación (0.2): intercambia dos entregas al azar para generar variedad.

Así, el algoritmo va generando y seleccionando rutas cada vez más eficientes.

6.4 Resultados

Como los dos algoritmos son aleatorios, corrimos cada uno con 5 semillas. La mejor distancia que encontramos en todos nuestros experimentos (incluido el de T_0) fue de 30,214 m, y la usamos como referencia para comparar.

Tabla 7. Resultados de SA y AG en 5 corridas (distancia en metros).

Semilla	SA	Tiempo SA	AG	Tiempo AG
0	31,704	65 ms	31,517	413 ms
1	30,552	41 ms	30,665	338 ms
2	30,552	79 ms	31,497	415 ms
3	30,845	49 ms	34,213	479 ms

Semilla	SA	Tiempo SA	AG	Tiempo AG
4	31,241	77 ms	32,391	368 ms
Promedio	30,979	62 ms	32,057	403 ms

Los dos redujeron la ruta ingenua de 61,835 m a la mitad. SA quedó en promedio a 2.5% de la mejor distancia conocida, y el AG a 6.1%. SA además evaluó unas 9,200 rutas contra 20,100 del AG y tardó unas 6 veces menos.

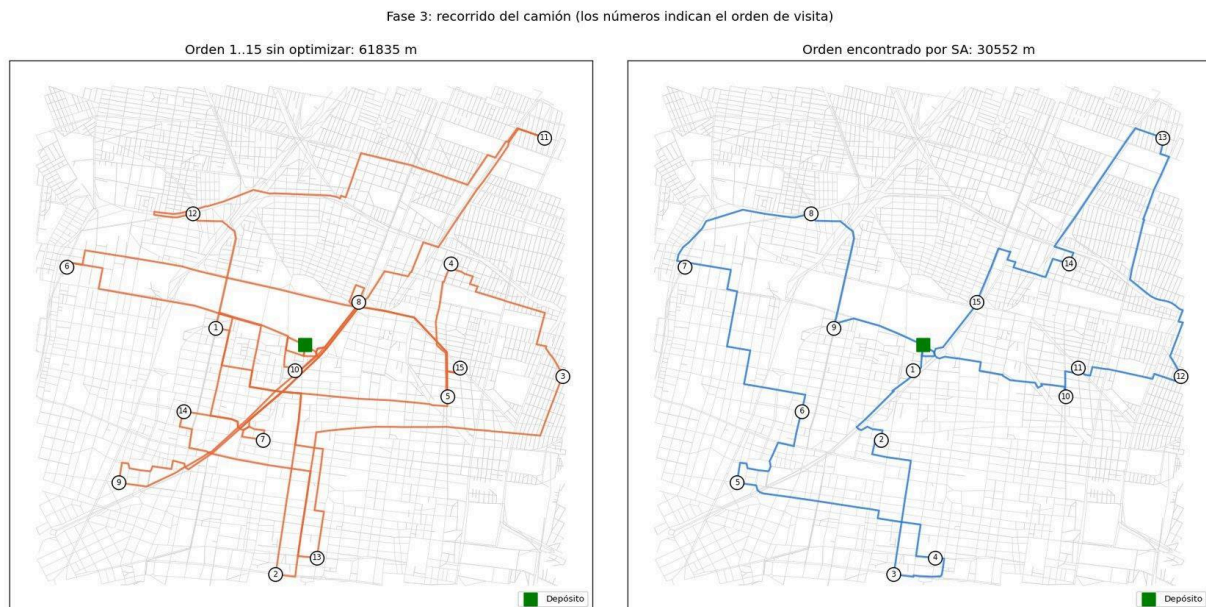


Figura 14. Recorrido del camión sobre las calles: orden 1..15 sin optimizar (izquierda) contra el orden encontrado por SA (derecha). Los números indican el orden de visita.

La Figura 14 muestra bien la diferencia: la ruta sin optimizar cruza el mapa de un lado a otro varias veces, mientras que la de SA hace un recorrido ordenado, primero por el sur y oeste y luego por el noreste, sin cruces.

6.5 Convergencia

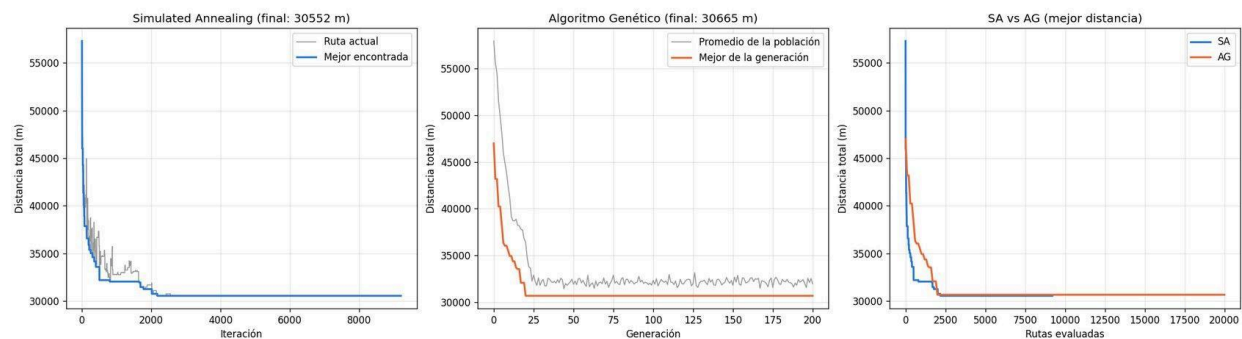


Figura 15. Convergencia de SA (izquierda), AG (centro) y comparación de ambos por rutas evaluadas (derecha).

En SA, la línea gris (ruta actual) sube y baja al principio porque acepta rutas peores, mientras la azul (mejor encontrada) solo baja. Después de unas 2,500 iteraciones ya no mejora y el resto

es enfriamiento. En el AG la mejor distancia baja muy rápido, pero alrededor de la generación 20 se estanca y el promedio de la población se queda pegado al mejor. Eso significa que casi todas las rutas de la población se parecen y ya no hay diversidad para seguir mejorando, lo que se conoce como **convergencia prematura**. El panel derecho compara a los dos por número de rutas evaluadas, que es lo justo porque el AG evalúa 100 rutas por generación y SA solo una por iteración.

6.6 Experimento de la temperatura inicial

Corrimos SA con $T_0 = 10, 100, 1000$ y 10000 , diez veces cada una. Para que fuera justo, todas hicieron las mismas 9,206 iteraciones; para eso ajustamos α con $\alpha = (T_{\min} / T_0)^{(1/N)}$. Si no, la T_0 más alta tendría más iteraciones y la comparación estaría sesgada.

Tabla 8. Experimento de T_0 (10 corridas por valor, 9,206 iteraciones cada una).

T_0	Mejor	Promedio	Peor	Desv. estándar
10	30,552	33,014	36,833	2,280
100	30,214	31,279	33,330	938
1000	30,552	30,975	31,704	382
10000	30,214	30,960	31,815	474

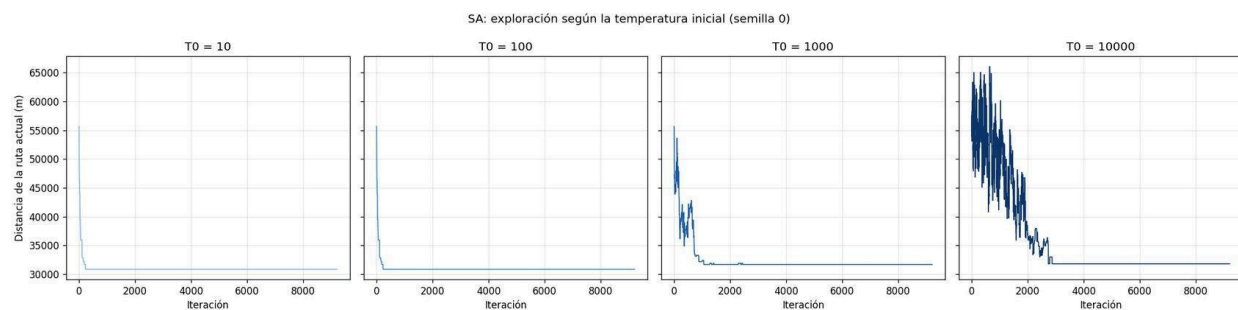


Figura 16. Distancia de la ruta actual de SA con cada T_0 (semilla 0).

6.7 Preguntas de la Fase 3

¿Por qué no se puede aplicar A^* para decidir el orden de 15 entregas? ¿De qué tamaño usas el espacio de estados?

Porque existen $15! = 1.3$ billones de rutas posibles. A^* tendría que manejar aproximadamente $15 \times 2^{15} \approx 491$ mil estados, consumiendo mucha memoria y tiempo. Por eso usamos SA y AG, que exploran solo una pequeña parte de las rutas posibles.

¿Cómo afecta la temperatura inicial T_0 del SA a la diversidad de soluciones?

Una T_0 baja acepta pocas rutas peores y puede quedar atrapada en mínimos locales. Una T_0 alta permite explorar más y escapar de ellos, aunque al principio realiza movimientos casi aleatorios. En nuestras pruebas, temperaturas altas dieron resultados más estables.

¿Qué es mejor en calidad/tiempo, el Genético o el Simulated Annealing?

En nuestras pruebas, SA obtuvo mejores resultados: 30,979 m promedio frente a 32,057 m del AG, y fue aproximadamente 6 veces más rápido. El AG presentó convergencia prematura, aunque podría mejorar ajustando sus parámetros.

7. Pruebas unitarias

Para las pruebas se realizaron 28 pruebas en total con pytest, divididas en test_fase1.py, test_fase2.py y test_fase3.py. En vez de meterle al mapa real de plano, armamos grafos y matrices chiquitas a mano donde ya sabíamos cuál era la respuesta correcta, para no batallar. Así las pruebas corren rapidísimo y no dependen de que esté el archivo del mapa.

Tabla 9. Resumen de las pruebas unitarias.

Archivo	Qué se prueba
test_fase1.py (8) BFS y UCS/DFS son búsquedas "a ciegas"	Se diseñó un grafo de 6 nodos para que BFS y UCS den caminos distintos (A-B-D con 2 saltos y 11 m contra A-C-E-D con 3 saltos y 3 m). Se probó DFS con un ciclo, destinos inalcanzables, origen igual a destino, formato común y accesibilidad.
test_fase2.py (10) Greedy y A* usan el mapa para guiarse	Se diseñó un grafo de 6 nodos con coordenadas reales donde Greedy cae en una trampa (2,000 m) y A* encuentra el óptimo (1,500 m). Como Greedy solo mira que tan cerca se ve(h(n)) y no lo que ya recorrió (g(n)), cae en esa trampa, mientras A* sí la evita porque toma en cuenta ambas cosas.
test_fase3.py (10) SA/AG no buscan un camino sino el mejor orden de varias entregas	Simulamos un deposito con 4 paradas de entrega, todas ubicadas en línea recta, para que sea facil saber cual es la ruta más corta posible (800 metros). Usamos una tabla de distancias donde ir de un punto a otro no cuesta lo mismo en ambos sentidos. Con esto comprobamos que las técnicas para mejorar rutas siempre generen recorridos válidos, que los dos métodos de optimización (SA y AG) logren encontrar esa ruta más corta, y que cuando vayan agarrando diferentes opciones, nunca terminen con una ruta peor de la que ya habían encontrado antes.

```
(.venv) PS C:\Inteligencia Artificial ESCOM\Prac1> pytest -v
===== test session starts =====
platform win32 -- Python 3.12.6, pytest-9.1.1, pluggy-1.6.0 -- C:\Inteligencia Artificial ESCOM\Prac1\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Inteligencia Artificial ESCOM\Prac1
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collected 28 items

tests/test_fase1.py::test_bfs_encuentra_menos_saltos PASSED [ 3%]
tests/test_fase1.py::test_ucs_encuentra_menos_metros PASSED [ 7%]
tests/test_fase1.py::test_dfs_encuentra_camino_valido PASSED [ 10%]
tests/test_fase1.py::test_destino_inalcanzable_regresa_none PASSED [ 14%]
tests/test_fase1.py::test_origen_igual_a_destino PASSED [ 17%]
tests/test_fase1.py::test_resultado_tiene_formato_comun PASSED [ 21%]
tests/test_fase1.py::test_nodos_alcanzables PASSED [ 25%]
tests/test_fase1.py::test_verificar_entregas PASSED [ 28%]
tests/test_fase2.py::test_heuristicas_miden_metros PASSED [ 32%]
tests/test_fase2.py::test_h1_y_h2_casi_iguales_en_distancias_cortas PASSED [ 35%]
tests/test_fase2.py::test_heuristicas_valen_cero_en_el_destino PASSED [ 39%]
tests/test_fase2.py::test_h3_sin_giro_cuando_esta_alineado PASSED [ 42%]
tests/test_fase2.py::test_h3_penaliza_cuando_esta_en_diagonal PASSED [ 46%]
tests/test_fase2.py::test_a_estrella_encuentra_el_optimo PASSED [ 50%]
tests/test_fase2.py::test_a_estrella_da_lo_mismo_que_ucs PASSED [ 53%]
tests/test_fase2.py::test_greedy_falla_en_este_grafo PASSED [ 57%]
tests/test_fase2.py::test_inalcanzable_regresa_none PASSED [ 60%]
tests/test_fase2.py::test_formato_igual_al_de_fase1 PASSED [ 64%]
tests/test_fase3.py::test_distancia_ruta_incluye_salida_y_regreso PASSED [ 67%]
tests/test_fase3.py::test_distancia_ruta_respeto_matriz_asimetrica PASSED [ 71%]
tests/test_fase3.py::test_2opt_genera_permutacion_valida PASSED [ 75%]
tests/test_fase3.py::test_2opt_invierte_un_tramo PASSED [ 78%]
tests/test_fase3.py::test_cruza_o_genera_permutacion_valida PASSED [ 82%]
tests/test_fase3.py::test_mutacion_intercambia_exactamente_dos PASSED [ 85%]
tests/test_fase3.py::test_sa_encuentra_el_optimo_en_la_linea PASSED [ 89%]
tests/test_fase3.py::test_ag_encuentra_el_optimo_en_la_linea PASSED [ 92%]
tests/test_fase3.py::test_historial_mejor_nunca_empeora PASSED [ 96%]
tests/test_fase3.py::test_elegir_entregas_solo_ida_y_vuelta PASSED [100%]

===== 28 passed in 3.28s =====
(.venv) PS C:\Inteligencia Artificial ESCOM\Prac1>
```

Figura 17. Las 28 pruebas pasan.

8. Conclusiones

Trabajar sobre un mapa real hizo que los resultados fueran mucho más claros que en los ejemplos de clase. Ver que DFS arma una ruta de 135 km para ir a Bellas Artes, o que Greedy se mete a la colonia Guerrero en vez de tomar Reforma, deja muy claro por qué importan las garantías de cada algoritmo.

De la Fase 1 nos quedamos con que la elección del algoritmo depende de qué se quiere minimizar: BFS minimiza calles y UCS minimiza metros, y para un camión lo que importa son los metros. De la Fase 2, que una buena heurística cambia todo: A* encontró los mismos caminos óptimos que UCS, expandiendo hasta 33 veces menos nodos. También vimos en números qué significa que una heurística no sea admisible: h3 sobreestimó solo en 0.11% de los casos y eso bastó para que A* perdiera 10 m en un par. Y aprendimos a no confiar ciegamente en lo que dice la teoría sin medirlo, porque la ventaja de Haversine sobre la euclidiana simplemente no aparece a esta escala.

En la Fase 3 vimos que para problemas combinatorios como el orden de entregas no tiene sentido buscar exhaustivamente, y que métodos aproximados como SA dan resultados muy buenos (a 1-2% de la mejor solución conocida) en milisegundos. También nos quedó claro que el AG necesita más ajuste de parámetros para competir y que la temperatura inicial controla qué tanto explora SA.

Como mejoras a futuro, se podrían ajustar los parámetros del AG para evitar la convergencia prematura, considerar tiempos de recorrido en lugar de solo distancia (tráfico, semáforos, tipo de calle) y rediseñar h3 para que la penalización por giros sea admisible, por ejemplo, agregando el costo de los giros al costo real.

9. Referencias

- Russell, S. y Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4a ed.). Pearson.
- Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 65, 126-139.
- Hagberg, A., Schult, D. y Swart, P. (2008). Exploring network structure, dynamics, and function using NetworkX. *Proceedings of the 7th Python in Science Conference*, 11-15.
- Hart, P. E., Nilsson, N. J. y Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100-107.
- Kirkpatrick, S., Gelatt, C. D. y Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671-680.
- Davis, L. (1985). Applying adaptive algorithms to epistatic domains. *Proceedings of the 9th International Joint Conference on Artificial Intelligence*, 162-164.
- Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research*, 6(6), 791-812.
- OpenStreetMap contributors (2026). Datos de mapas de la Ciudad de México.
<https://www.openstreetmap.org>